Stage 2: Architectural Design Outline:

Community Healthcare Appointment & Referral Coordination Platform Domain

Table of Contents

Summary: 1
System Context (C4 Context) Diagram: 3
High-Level System Structure (C4 Container) Diagram: 4
Structural Rationale 5
Domain Structure (DDD thinking): 5
Workflow and Behaviour Modelling: 6
Consistency & Coordination Thinking: 7
Behavioural Reactions: 8
Failure Considerations: 8
Design Decision Summary: 9
Non-Functional Considerations 9

Summary:

The Community Healthcare Appointment & Referral Coordination Platform is designed to support coordination between healthcare providers involved in patient referrals and appointment scheduling. In many healthcare systems, referrals between primary care providers and specialist services involve fragmented communication channels such as phone calls, emails, or separate booking systems. This fragmentation leads to delays, lost referrals, poor visibility of referral status, and difficulties coordinating appointments.

The proposed system aims to provide a shared coordination platform that enables healthcare providers and patients to manage referrals and appointments through a structured workflow. The system supports communication between primary care providers, specialist clinics, diagnostic centres, and patients while integrating with external health record systems where necessary.

The system acts primarily as a **coordination platform**, rather than replacing existing clinical systems. External health record systems continue to manage patient medical data, while the coordination platform manages workflow state and interactions between actors.

Key Actors:

The platform involves several key actors who interact with the system in different ways.

Patients

- View referral status
- Book, confirm, reschedule, or cancel appointments
- Receive notifications about referral updates and appointments

Primary Care Providers (GP Clinics)

- Create referrals for specialist consultation or diagnostic services
- Provide relevant patient information
- Track referral progress and outcomes

Specialist Providers

- Receive and triage referrals
- Accept, reject, or request additional information
- Offer appointment availability
- Provide outcomes following appointments

External Partner Systems

- Electronic Health Record (EHR) systems providing patient data
- Notification services used to deliver messages and reminders

Core Workflows:

The system supports several multi-step workflows involving state changes over time.

Referral Initiation and Acceptance Workflow:

1. A GP creates a referral for a patient.
2. The referral is transmitted to a specialist provider.
3. The specialist triages the referral and may accept, reject, or request more information.
4. The patient and referring provider are notified of the outcome.
5. Accepted referrals move to the appointment scheduling phase.

Appointment Booking and Lifecycle Workflow

1. Providers publish appointment slots.
2. Patients select or are assigned appointment times.
3. Appointment confirmations are sent.
4. Patients may cancel or reschedule appointments.
5. Appointment status updates occur after completion.

Referral Follow-Up Workflow

1. Appointment outcomes are recorded.
2. Outcomes are shared with the referring provider.
3. The patient may receive follow-up instructions.
4. Additional referrals or treatment workflows may begin.

Important Business Rules:

Several domain rules shape system behaviour:

- A referral must be triaged **before** an appointment can be scheduled.
- Patients may hold only one active appointment **per referral**.
- Appointment slots cannot be double-booked.
- Referrals missing required clinical information must be updated before triage.
- Appointment cancellation inside a defined window *may* trigger re-triage.

These rules enforce domain consistency while enabling coordination between providers.

System Context (C4 Context) Diagram:

The system context diagram illustrates how the platform interacts with users and external systems.

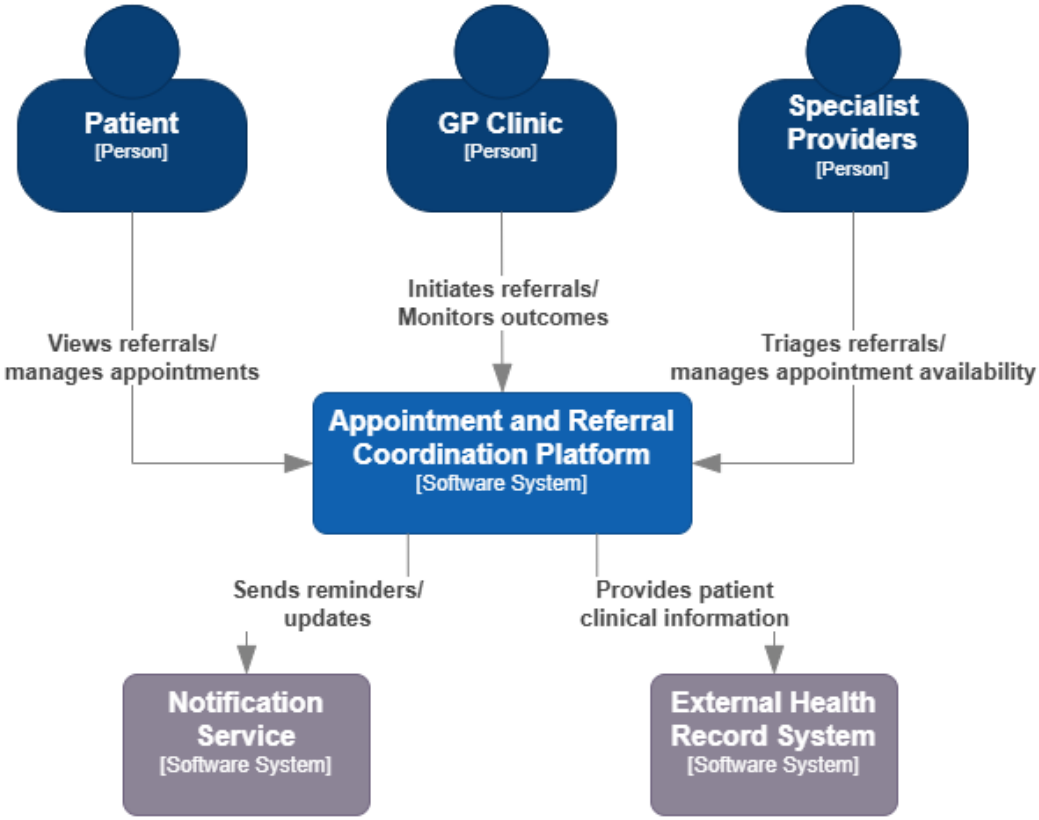

Figure 1: Patient Care Coordination System - Context Diagram

Explanation

The coordination platform acts as the **central system boundary** connecting several actors.

- **Patients** interact with the system to view referrals and manage appointments.
- **GP Clinics** initiate referrals and monitor outcomes.
- **Specialist Providers** triage referrals and manage appointment availability.
- **External EHR systems** provide patient clinical information.
- **Notification services** deliver reminders and updates.

The system functions as a coordination hub rather than a full clinical system, ensuring separation of responsibilities.

High-Level System Structure (C4 Container) Diagram:

The internal structure of the system is organised into containers representing major functional responsibilities. These containers separate user interaction, referral management, scheduling operations, notification processing, and integration with external systems. This separation allows the platform to coordinate complex workflows between healthcare providers while maintaining clear boundaries between core domain processes and supporting capabilities.

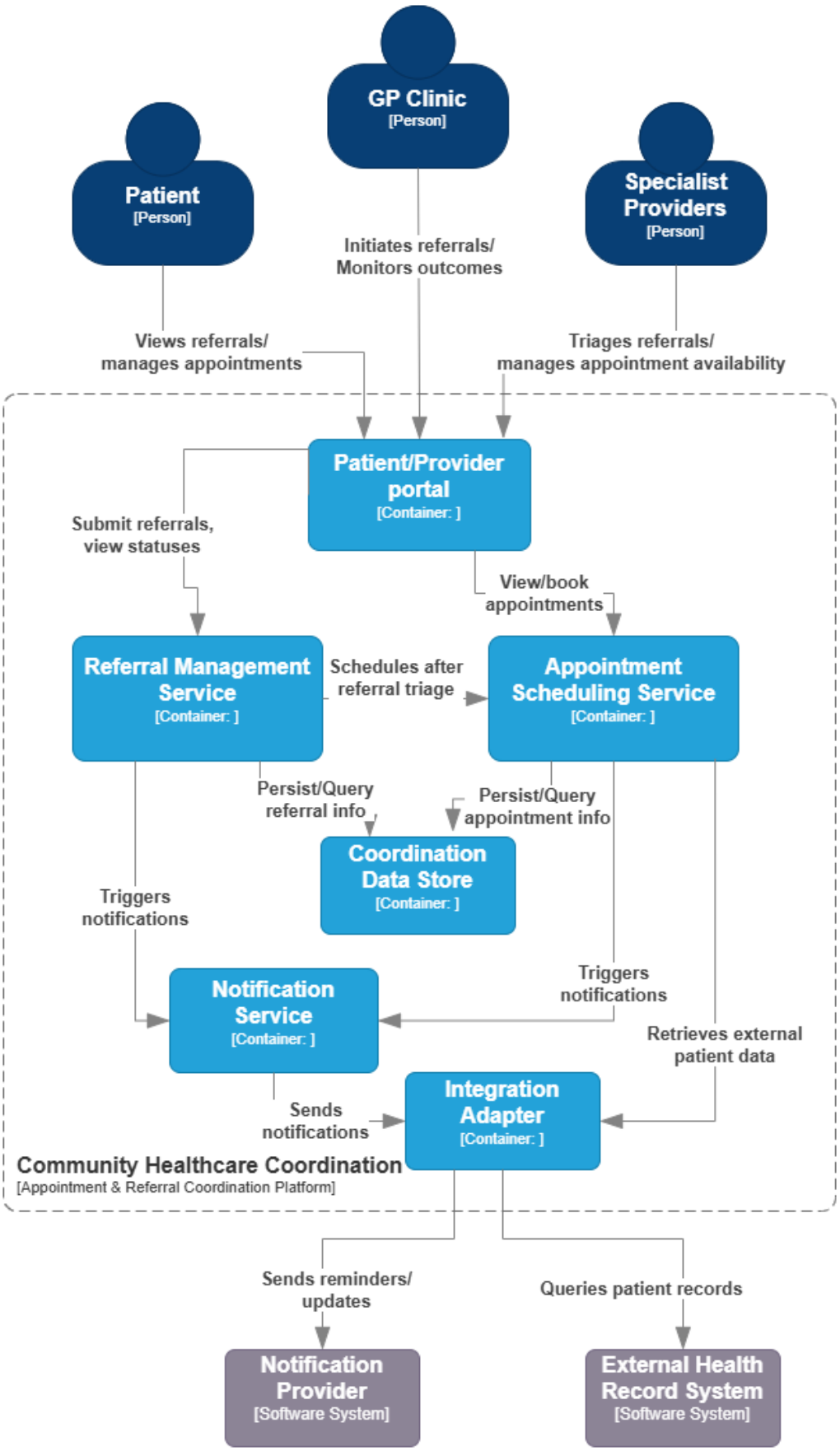

Figure 2: Patient Care Coordination System - Container Diagram

Container Responsibilities

User Interaction Container

Provides the system interfaces used by patients, GP clinics, and specialist providers. It handles authentication, displays referral and appointment information, and allows users to initiate actions such as referral creation or appointment booking. Separation of this container from domain services ensures that presentation concerns remain independent from workflow logic and allows the user interface to evolve without affecting core services.

Referral Management Service

Manages the referral lifecycle, including referral creation, validation, status tracking, and specialist triage decisions.

Appointment Scheduling Service

Handles appointment slot publication, booking, cancellation, and rescheduling while enforcing scheduling constraints such as preventing double booking.

Notification Service

Generates notifications triggered by system events, such as referral acceptance or appointment confirmations. The Notification Service is separated because communication events occur frequently and across multiple workflows. Decoupling notification generation ensures that operational processes do not depend on the success of message delivery.

Integration Adapter

Handles communication with external systems including External Health Record (EHR) platforms and notification providers. This container represents the system's **integration boundary** and **isolates internal services from external APIs and protocols**. The Integration Adapter container isolates communication with external systems. External systems are outside the control of the platform and may change independently. By isolating integration logic, domain services remain independent from external APIs and protocols.

Coordination Data Store

Persists referral and appointment information required for long-running workflows. The database stores durable state such as referral status, appointment bookings, and scheduling availability.

Clarification on Persistence Access

The Coordination Data Store is accessed directly by the domain services rather than through the Integration Adapter as the database is infrastructure owned by the platform. The Integration Adapter is reserved for interactions that **cross the system boundary**, such as communication with external healthcare systems or messaging providers, so it is not involved.

Structural Rationale

The system separates responsibilities into logical containers to improve maintainability and clarity of domain responsibilities. Referral management and appointment scheduling represent different stages of the patient care coordination process and therefore have distinct responsibilities. Separating them allows rules such as requiring referral triage before scheduling while keeping scheduling logic independent.

Although the system is structured into multiple containers, the design does not assume a microservice deployment architecture. The containers represent logical **responsibility boundaries** rather than independent *deployment units*. In practice, the platform could be implemented as a modular application containing these components while still maintaining clear architectural separation. This keeps operational complexity low while preserving the ability to evolve toward more distributed architectures if system scale increases.

Domain Structure (DDD thinking):

The domain can be conceptually divided into several core areas that represent the main coordination responsibilities of the platform. Each area protects a set of responsibilities and rules that must remain consistent during workflow execution.

Referral Domain

The Referral domain manages the lifecycle of referrals created by GP clinics and processed by specialist providers. A natural aggregate boundary in this area is the **Referral**, which encapsulates information such as referral details, patient identifiers, and triage status. The referral aggregate protects the integrity of the referral lifecycle and ensures that transitions between states occur in a valid order.

Important lifecycle states include **Created**, **Pending Triage**, **Information Requested**, **Accepted**, and **Rejected**. The referral aggregate ensures that only valid state transitions occur, for example preventing appointment scheduling before a referral has been accepted. It also protects rules such as **requiring mandatory information before triage can proceed,** as well as guaranteeing a **referral's state transitions follow the defined lifecycle**. Responsibilities within this boundary include validating referral data, tracking status changes, and coordinating communication with specialist providers.

Appointment Domain

The Appointment domain is responsible for scheduling and managing appointments that occur after a referral has been accepted. The main aggregate boundary is the **Appointment**, which encapsulates booking details, appointment status, and associated referral identifiers.

This aggregate protects scheduling rules and ensures that appointment-related invariants are upheld. Important invariants include **preventing double booking of appointment slots, ensuring that each referral has at most one active appointment at a time, and enforcing valid lifecycle states** such as **Scheduled**, **Confirmed**, **Cancelled**, and **Completed**. Responsibilities within this boundary include publishing appointment availability, managing bookings, and handling rescheduling or cancellation requests.

Notification Domain

The Notification domain manages system-generated communications triggered by workflow events. A notification aggregate represents a message generated in response to a domain event such as referral acceptance or appointment confirmation.

The boundary protects the responsibility of determining when notifications should be generated and what information they contain. An important invariant is that **notifications should only be generated in response to valid domain events and must reference existing referral or appointment data**. This domain ensures that communication logic remains separate from core workflow processing while still reacting to important system state changes.

Together, these domain areas encapsulate the primary coordination responsibilities of the platform and ensure that important workflow rules are enforced consistently throughout the system.

Workflow and Behaviour Modelling:

Chosen workflow: Referral Initiation and Acceptance

The referral initiation and acceptance workflow begins when a **GP clinic** creates a referral request through the **portal (user interaction container)**. The portal forwards the request to the **Referral Management Service**, which validates the referral information and stores it in the **Coordination Data Store**. At this point the referral enters an initial state such as ***Created*** or ***Pending Triage***.

A **specialist provider** then accesses the system through the same portal to review referrals awaiting triage. The portal retrieves the relevant referral information from the referral service, which reads the stored referral data from the coordination data store. After reviewing the referral details, the specialist provider submits a triage decision through the portal. This decision may accept the referral, reject it, or request additional information. The portal forwards the decision to the referral service, which updates the referral status and persists the new state in the coordination data store.

Once the referral status changes, the referral service triggers a notification event. The **Notification Service** prepares a message indicating the updated referral status and passes it to the **Integration Adapter**, which communicates with an external notification provider to deliver the update to the **patient**. If the referral has been accepted, the patient can proceed to the **appointment scheduling workflow**, where appointment slots can be viewed and booked.

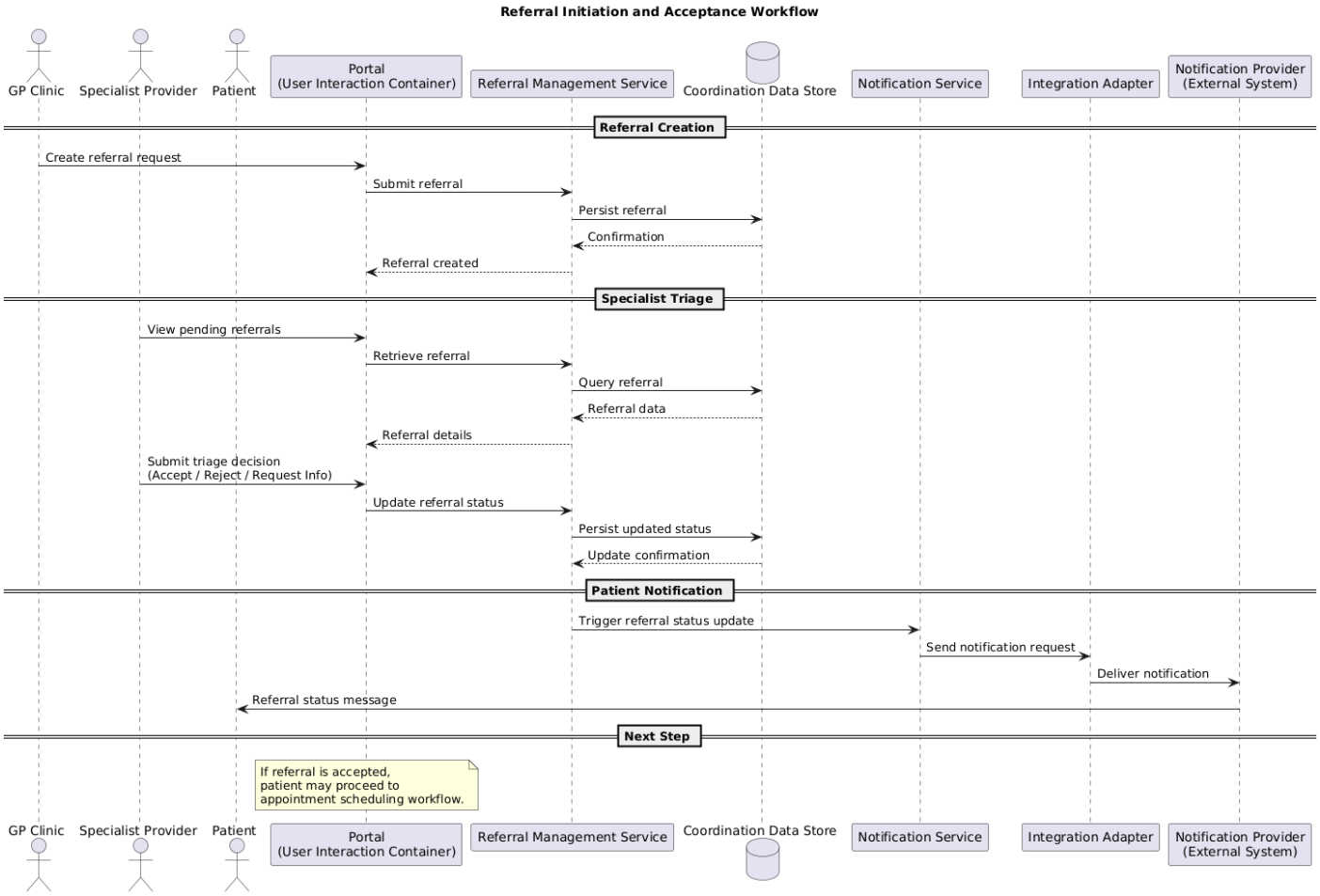

Figure 3: Patient Care Coordination System - Referral Initiation and Acceptance Workflow

Failure Handling

If the workflow does not complete successfully, the referral remains in an intermediate state rather than progressing to scheduling. For example, a specialist provider may reject the referral or request additional information if the referral details are incomplete. In this case the referral status is updated accordingly, and the referring GP is notified so the information can be corrected and resubmitted. Notification delivery may also fail due to external messaging issues; however, this does not affect the stored referral state, allowing the system to retry message delivery without disrupting the referral process.

Consistency & Coordination Thinking:

The system manages workflows that span multiple domain areas, meaning some operations require strong consistency while others can tolerate temporary inconsistency. Identifying where strict consistency is required helps ensure that important domain rules are always upheld while allowing flexibility in less critical areas.

Where consistency is critical

Consistency is essential in situations where violating a rule would result in an invalid or conflicting system state.

One example is **appointment scheduling**, where appointment slots must not be double-booked. When an appointment is created or updated, the scheduling service must ensure that a specific time slot is allocated to only one patient. This rule must be enforced immediately to prevent conflicting bookings.

Another example is **referral lifecycle transitions**. A referral must move through valid lifecycle states, such as ***Created***, ***Pending Triage***, and ***Accepted*** or ***Rejected***. Invalid state transitions must never occur, as they could allow workflows to proceed incorrectly or bypass required review steps.

Where temporary inconsistency may be acceptable

Some system behaviours can tolerate temporary inconsistency without affecting overall correctness. For example, **notification delivery** may occur asynchronously after a referral status update or appointment

confirmation. If notification delivery is delayed or temporarily fails, the core referral and appointment data remain correct within the coordination data store.

Similarly, **external system integrations**, such as retrieving patient information from an external health record system, may occasionally experience delays. In these cases, the system can continue processing workflows while synchronizing external data later.

Coordination reactions

Certain events require coordinated actions across multiple components. For example, when a **referral is accepted**, the system must update the referral status, notify the patient, and allow the patient to proceed to appointment scheduling. These reactions require coordination between the referral service, notification service, and scheduling components while maintaining consistent workflow progression.

Behavioural Reactions:

The system contains several situations where a change in one part of the platform triggers activity in other components. These reactions support coordination between services while maintaining clear responsibility boundaries.

One example occurs when a **referral is triaged and accepted by a specialist provider**. The referral service updates the referral status in the coordination data store to reflect the new lifecycle state. This change triggers several follow-up reactions. The notification service generates a message informing the patient that the referral has been accepted, which is delivered through the integration adapter to an external messaging provider. At the same time, the system allows the patient to proceed to the appointment scheduling workflow through the portal. Decoupling these reactions from the referral service is beneficial because it keeps the referral domain focused on managing referral state rather than communication or scheduling logic. This separation allows notification or scheduling behaviour to evolve independently without affecting the referral lifecycle.

A second example occurs when an **appointment is successfully scheduled**. The appointment scheduling service records the booking in the coordination data store and updates the appointment lifecycle state to ***Scheduled***. This change triggers the generation of confirmation notifications for the patient and potentially the referring provider. The notification service prepares these messages and sends them through the integration adapter for delivery. Additional reactions may include updating referral progress or preparing follow-up workflows once the appointment has been completed. Decoupling these reactions ensures that the scheduling service remains responsible only for enforcing scheduling rules such as preventing double bookings, while other services handle communication and downstream workflow activities.

By separating these reactions from the originating services, the system reduces coupling between components and supports more flexible evolution of workflows and integrations.

Failure Considerations:

The system must account for realistic failure scenarios that may interrupt workflows or delay coordination between actors. Designing for these situations helps ensure that failures in one part of the system do not compromise the overall integrity of referral and scheduling processes.

One potential failure occurs when **referral information is incomplete or invalid at submission time**. A GP clinic may create a referral that lacks required clinical details or supporting documentation. In this case the referral service cannot move the referral to the ***Pending Triage*** state and the workflow cannot proceed to specialist review. The impact is that the referral remains in a preliminary state until the information is corrected. Other components, such as the *scheduling service*, are unaffected because appointment scheduling cannot occur until the referral reaches an accepted state. The design reduces the impact of this failure by validating referral information within the referral service before persistence and by allowing the referral to remain in an intermediate state until the GP updates the information.

Another realistic failure scenario involves **external notification delivery failures**. When a referral status changes or an appointment is scheduled, the notification service attempts to send messages through the integration adapter to an external messaging provider. If the external provider is temporarily unavailable, the notification may fail to deliver. This failure does not affect the core workflow because the referral and

appointment states have already been recorded in the coordination data store. The system design reduces the impact of this failure by decoupling notification delivery from domain services. Since notifications are triggered asynchronously, the referral and scheduling workflows can complete successfully while notification delivery can be retried later without affecting system consistency.

Design Decision Summary:

The system architecture was structured around clear domain responsibilities in order to support the coordination workflows involved in managing referrals and appointments across healthcare providers. The design separates the system into containers representing user interaction, referral management, appointment scheduling, notifications, integration with external systems, and persistent coordination data. This structure reflects the natural boundaries of the domain and allows each component to focus on a specific responsibility. For example, the referral service manages referral lifecycle state while the scheduling service enforces appointment rules such as preventing double booking. Separating these concerns helps maintain clarity and maintainability as workflows evolve.

Several alternative approaches were considered during the design process. One option was to structure the system as a distributed microservice architecture, where each domain area would be independently deployed and managed. However, this would introduce operational complexity that is not necessary for a system of this scale. Instead, the design treats the containers as logical boundaries within a potentially modular application. This preserves separation of responsibilities while keeping the system simpler to deploy and maintain.

Another design choice involved isolating communication with external systems through an integration adapter. This approach protects the core domain services from changes in external APIs and messaging providers, although it introduces an additional layer in the interaction flow.

Some uncertainty remains around how the system may evolve as integration requirements grow. For example, additional external healthcare systems or provider organisations may require further integration capabilities. If the platform grows in complexity or usage volume, the current modular architecture could evolve toward more independently deployable services while preserving the same domain boundaries.

Non-Functional Considerations

Several design decisions in the architecture also support important non-functional qualities. The separation of responsibilities between referral management, scheduling, notifications, and integration components helps improve **maintainability**, as each part of the system focuses on a clear domain concern and can evolve without affecting unrelated functionality. The inclusion of a coordination data store ensures **reliability** by maintaining durable workflow state for referrals and appointments even when other components or external systems experience temporary failures. Isolating external interactions through the integration adapter supports **evolvability**, allowing the system to adapt more easily to changes in external healthcare systems or messaging providers. Finally, structuring the system into logical containers allows the architecture to support **scalability** if needed in the future, as individual components could be deployed or scaled independently as system usage grows.